Software Engineering & Project Management (SEPM)

Energy & Utility Management System (EUMS)

Billing, metering and customer service for electricity, water and gas

Software Requirements Specification

Software process model: Waterfall · Combined SEPM + DBMS course project

Group number	Group __
Course and section	Software Engineering & Project Management (SEPM), Section __
Supervisor / instructor	Prof. _____
Institute	_____ Institute of Technology
Date	October 2026

#	Name	Registration number
1	Sumit P Surendran	20243590
2	Sreethan Reddy	20243550
3	Sushrut Lad	20243551
4	Viswajith	20243559

Document	Software Requirements Specification (IEEE 830 structure)
Version	1.0 (baselined at end of Requirements phase; status columns updated at mid-semester)
Product	Energy & Utility Management System (EUMS)

1. Introduction

1.1 Purpose

This document specifies the requirements of the Energy & Utility Management System (EUMS). It is the agreed baseline between the development team and the course instructor, and it is the input to the design, implementation and testing phases of our Waterfall process.

1.2 Scope

EUMS is a web application for a utility office that supplies electricity, water and piped gas. It manages customers, connections and meters, records meter readings, generates slab-based bills automatically, accepts payments, and handles complaints and outages. It provides a staff portal (administrator and employees) and a customer self-service portal. Real payment gateways, smart meters, SMS/e-mail and mobile apps are out of scope.

1.3 Definitions and abbreviations

Term	Meaning
Connection	A customer's supply of one utility at their address
Meter reading	The cumulative value shown on a meter on a date
Consumption	Current reading − previous reading
Slab tariff	Progressive pricing: each band of units has its own rate
Fixed charge	Monthly charge independent of usage
Outstanding	Bill total − amount already paid
kWh / kL / SCM	Units for electricity / water (kilolitre) / gas (standard cubic metre)
RBAC	Role-based access control
MoSCoW	Must / Should / Could / Won't prioritisation

1.4 References

IEEE Std 830-1998; course brief for the DBMS and SEPM projects; state electricity tariff orders (LMV-1/LMV-2) used as a model for slab structure.

1.5 Overview

Section 2 describes the product in general terms. Section 3 lists interfaces. Sections 4–6 give the functional, non-functional and data requirements.

2. Overall Description

2.1 Product perspective

EUMS is a new, self-contained system that replaces paper registers and spreadsheets. It is a three-tier web application (browser, Flask server, SQLite database).

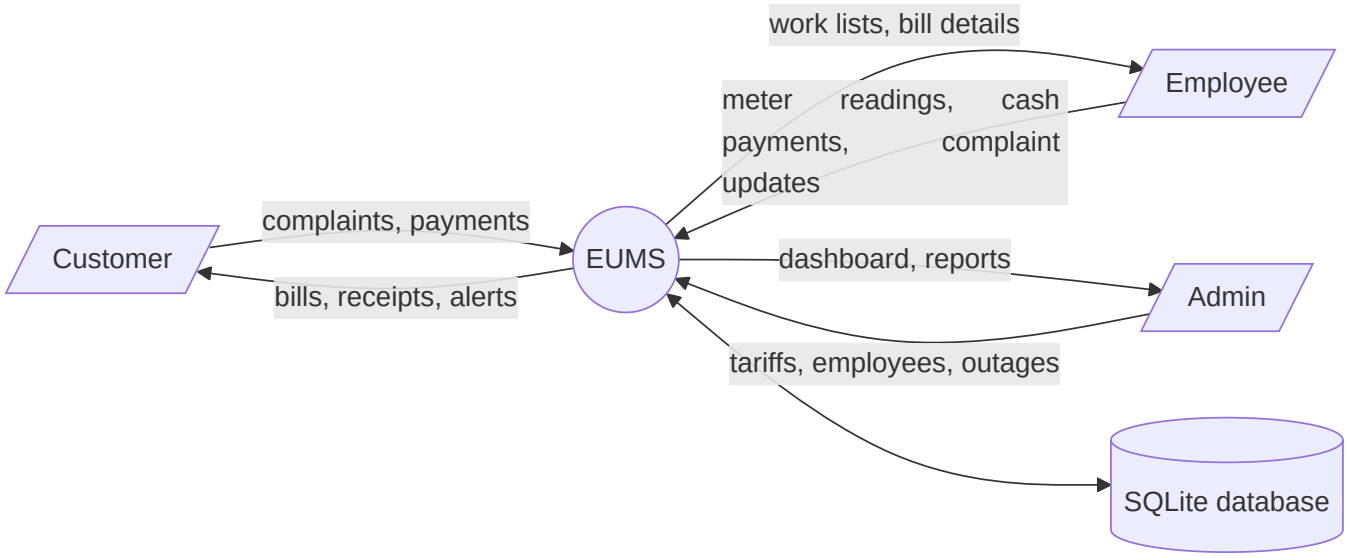


Figure 1: Context diagram

2.2 Product functions

- Manage users, customers, employees, connections and meters.
- Record meter readings with validation; generate itemised slab-based bills.
- Accept online and counter payments, including partial payments, and issue receipts.
- Manage complaints and outages; notify customers.
- Maintain tariffs; show dashboards and reports.

2.3 User classes and characteristics

User class	Characteristics	Access
Administrator	Office manager; moderate computer skills	Everything, including tariffs, employees, reports
Employee	Meter reader, billing officer or technician; basic skills; may use a phone in the field	Customers, connections, readings, billing, payments, complaints, outages
Customer	General public, residential or commercial; uses UPI apps	Only their own connections, bills, usage, complaints, outages in their area

2.4 Operating environment

Server: any PC with Python 3.10+ and Flask (Windows, Linux or macOS). Client: any modern browser (Chrome, Edge, Firefox) on desktop or phone. Database: SQLite 3 file.

2.5 Design and implementation constraints

- Free and open-source software only; must run on a student laptop without a database server.
- Relational database with constraints, triggers and views (DBMS course requirement).
- No passwords, API keys or real personal data in the source code or sample data.

2.6 Assumptions and dependencies

- One connection per utility per customer; one active meter per connection; meters are cumulative.
- Tariff depends only on the utility and the customer category (residential / commercial).
- Bills are due 15 days after the bill date. Payment gateway is simulated.

3. External Interface Requirements

3.1 User interfaces

Web pages with a left navigation menu by role, a top bar with the user's name, cards for summaries, tables for lists and simple forms. Errors are shown as red messages at the top of the page; successes in green. Layout adapts to screens narrower than 800 px.

3.2 Software interfaces

Python Flask 3 web framework; SQLite 3 via Python's `sqlite3` module; Chart.js 4 in the browser. A JSON endpoint `GET /staff/api/meters/<id>` returns the previous reading for the reading form.

3.3 Communication interfaces

HTTP between browser and server (HTTPS in a production deployment). Session cookie signed with a secret key.

4. Functional Requirements

ID	Area	Requirement	Priority	Status
FR-01	Login	Users log in with username/email and password; passwords are stored hashed.	Must	Done
FR-02	Access	Each page only opens for its roles; others get "403", and logged-out users go to the login page.	Must	Done
FR-03	Customers	Staff add a customer (name, unique email, 10-digit phone, address, area, category, password).	Must	Done
FR-04	Customers	Staff search customers and open a profile with their connections and bills.	Should	Done
FR-05	Connections	Staff add a connection; the tariff is picked from the category and a meter is created. One per utility per customer.	Must	Done
FR-06	Readings	Picking a meter shows the customer, utility and last reading; consumption is shown while typing.	Must	Done
FR-07	Readings	Readings lower than the last one, or not after its date, are refused.	Must	Done
FR-08	Billing	Saving a reading creates the bill: slab charges + fixed charge + tax, itemised, due in 15 days.	Must	Done
FR-09	Billing	Staff list and filter bills; anyone allowed can open a bill and print it.	Must	Done
FR-10	Payments	Customers pay online by card, UPI, net banking or wallet (simulated), fully or in part, and get a receipt number.	Must	Done
FR-11	Payments	Paying more than is due is refused; the bill becomes partial or paid automatically.	Must	Done
FR-12	Payments	Staff record cash payments and see all payments with totals by method.	Should	Done
FR-13	Complaints	Customers raise complaints; staff assign, prioritise, update and resolve them.	Must	Done
FR-14	Outages	Staff report outages by utility and area (scheduled or active) and mark them resolved.	Should	Done
FR-15	Notifications	Customers get in-app notices for new bills, payments, outages in their area and complaint updates.	Should	Done

FR-16	Tariffs	Admin edits slab rates, fixed charge and tax; only new bills use the new values.	Must	Done
FR-17	Employees	Admin sees employees and their workload and can add new ones.	Should	Done
FR-18	Dashboards	Staff and customer dashboards with key numbers, charts and recent items.	Should	Done
FR-19	Reports	Usage per utility, by area and by category, collection rate and overdue bills.	Could	Partial
FR-20	Portal	Customers see connections, usage history and local outages, and edit phone, address and password.	Should	Done
FR-21	Billing	Late-payment charge and overdue reminders.	Could	Planned
FR-22	Billing	Server-generated PDF invoice and e-mail.	Could	Planned
FR-23	Meters	Replace a faulty meter and start the new one from its own reading.	Should	Planned
FR-24	Reports	Export reports to CSV/Excel.	Could	Planned
FR-25	Payments	Real payment gateway.	Won't	Planned

Detailed use-case descriptions and user stories with acceptance criteria are in `02_Requirements/Use_Cases_or_User_Stories.pdf`.

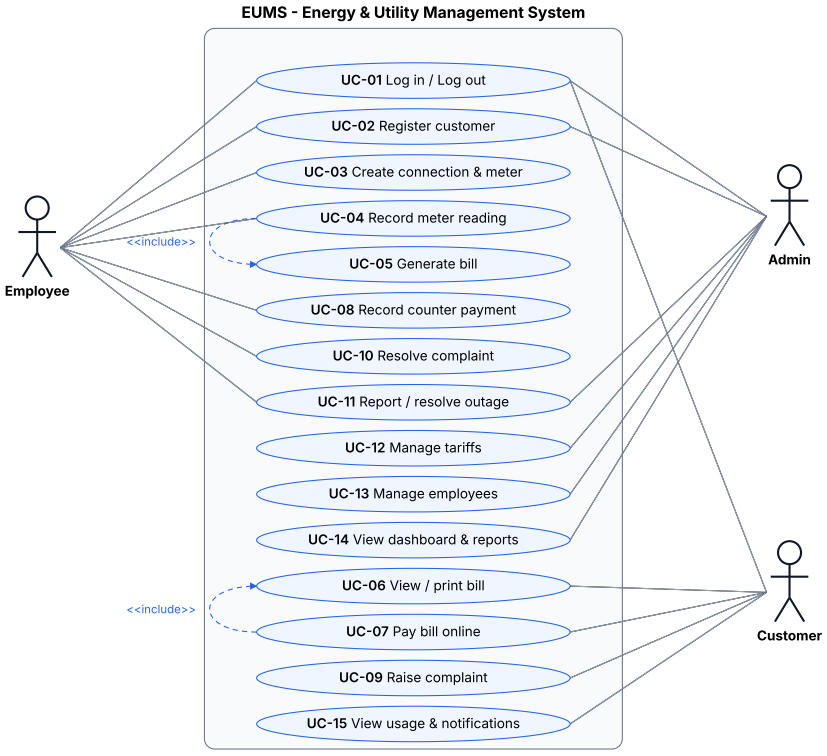


Figure 2: Use-case diagram

5. Non-Functional Requirements

ID	Type	Requirement	How we meet it
NFR-01	Security	Passwords are never stored as plain text	Salted hashes from Werkzeug
NFR-02	Privacy	Customers only see their own records	Queries filtered by the logged-in customer (test IT-05)

NFR-03	Security	No secrets in the code	Secret key in a git-ignored `config.py`
NFR-04	Integrity	Business rules hold even if a page has a bug	Constraints and 7 triggers in the database
NFR-05	Accuracy	Bills are correct to the paisa	Rounding to 2 decimals; unit tests
NFR-06	Auditability	Issued bills never change	Charges copied onto the bill (test IT-14)
NFR-07	Performance	Pages load in under a second with 1,000 customers	Indexes on the busy columns
NFR-08	Usability	A new employee can enter a reading without training	One task per page, live feedback, plain error messages
NFR-09	Portability	Runs on Windows, Linux and macOS	Python + Flask only; SQLite is built in
NFR-10	Mobile	Usable on a phone in the field	Layout adapts below 800 px
NFR-11	Reliability	A failed action leaves nothing half-saved	One transaction per action, rolled back on error

6. Data Requirements

The system stores the following entities: users, customers, employees, utilities, tariff plans and slabs, connections, meters, meter readings, bills and bill items, payments, complaints, outages, notifications and an activity log. Key business rules on data:

- BR-1: A meter reading must be ≥ the previous reading of the same meter, and later in date.
- BR-2: Exactly one bill per meter reading; bill charges are fixed once issued.
- BR-3: Total payments on a bill can never exceed the bill total. Status is unpaid, partial or paid.
- BR-4: Customer email, customer code, connection number, meter number, bill number and receipt number are unique.
- BR-5: A customer has at most one connection per utility.

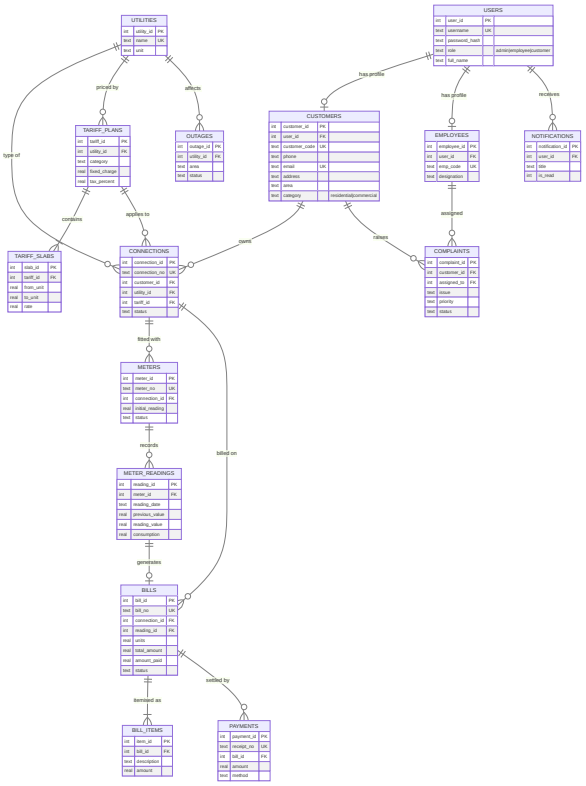


Figure 3: Logical data model (ER diagram)